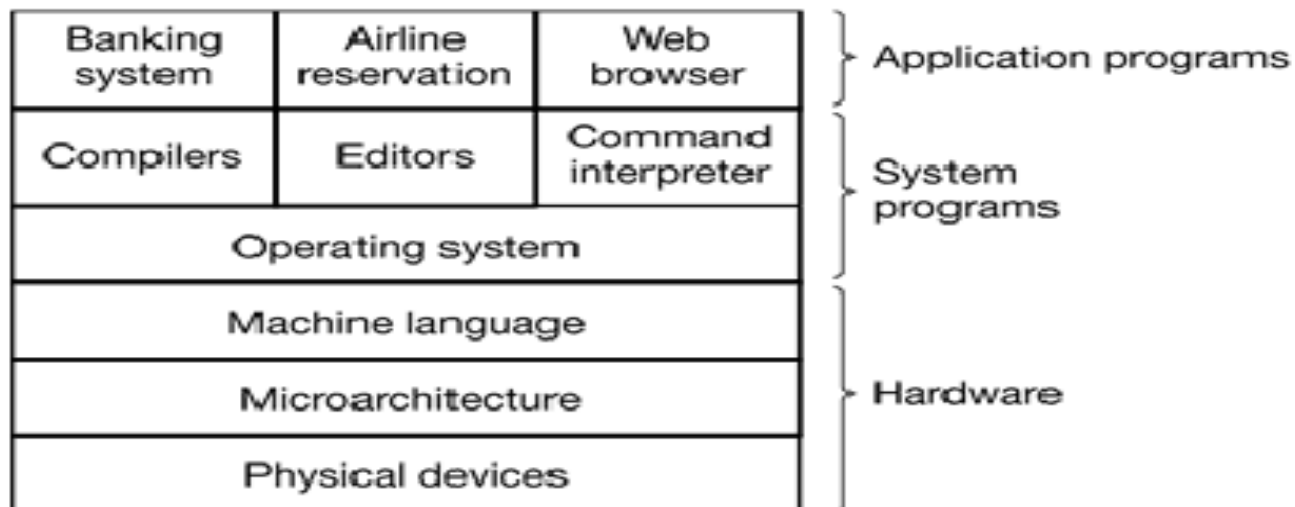


INTRODUCTION

Chapter 1

Introduction

- A modern computer system consists of one or more processors, some main memory, disks, printers, a keyboard, a display, network interfaces, and other input/output devices.
- To **keep track of all these components and use them correctly**, computers are equipped with a layer of software called the operating system.
- **Operating system manages all devices and provides user programs with a simpler interface to the hardware.**



- The lowest level in computer system contains physical devices consisting of integrated circuit chips, wires, power supplies, cathode ray tubes.
- Next level is the microarchitecture level containing
 - some registers internal to the CPU (Central Processing Unit) and
 - Data path containing an arithmetic logic unit.
- In each clock cycle, one or two operands are fetched from the registers and are combined in the arithmetic logic unit and later results are stored in one or more registers.
- Machine language layer is devoted to processing the machine languages. Machine language are limited set of instruction that directly influence the working of physical devices.
- Operating system consists of a layer of software that (partially) hides the hardware and gives the programmer a more convenient set of instructions to work with.
- On top of the operating system are the rest of the system software like command interpreter (shell), window systems, compilers, editors, and similar application-independent programs that are not a part of operating system.
- The operating system runs in kernel mode or supervisor mode while application programs run in user mode.

OPERATING SYSTEM AS AN EXTENDED MACHINE:

- In general, To make a system with set of physical devices work, systems architecture including instruction set, memory organization, I/O and bus structures must be known in-order to write the machine language that commands the physical devices to work.
- But due to the amount of complexity among different architectures and need to master the machine language that varies from one architecture to another, it became difficult for common peoples to access the system.
- Hence operating system was created whose function is to present the user with the equivalent of an extended machine or virtual machine that is easier to program than the underlying hardware.
- This is achieved by variety of services that programs can obtain using special instructions called system calls.

OPERATING SYSTEM AS A RESOURCE MANAGER:

- the job of the operating system is to provide for an orderly and controlled allocation of the processors, memories, and I/O devices among the various programs competing for them.
- As an example, what would happen if three programs running on some computer all tried to print their output simultaneously on the same printer. The first few lines of printout might be from program 1, the next few from program 2, then some from program 3, and so forth.
- The result would be chaos. So operating system buffers all the output destined for the printer on the disk. When one output is printed completely, then the next output stored is taken and printed.
- Similarly When a computer (or network) has multiple users, the need for managing and protecting the memory, I/O devices, and other resources is even greater, since the users might interfere with one another. In addition, users often need to share not only hardware, but information's like files, databases, etc. as well.

- So the operating system primary task is to keep track of
 - who is using which resource,
 - to grant resource requests,
 - to account for usage, and
 - to mediate conflicting requests from different programs and users.
- Sharing of resources can be done in two ways: in time or space.
- When a resource is time multiplexed, different programs or users wait for their turn to use the resources. An example of time multiplexing is sharing the printer. When multiple print jobs are queued up for printing on a single printer, a decision has to be made about which one is to be printed next.
- If the resources are space multiplexed, instead of the customers taking turns, each one gets part of the resource. For example, main memory is normally divided up among several running programs, so each one can be resident at the same time

1. HISTORY OF OPERATING SYSTEMS

- The first true digital computer was designed by the English mathematician Charles Babbage (1792-1871). Although Babbage spent most of his life and fortune trying to build his “analytical engine.” he never got it working properly because it was purely mechanical, and the technology of his day could not produce the required wheels, gears, and cogs to the high precision that he needed.

1.1. The First Generation (1945-55) Vacuum Tubes and Plug boards:

- Around the mid-1940s, many scientists succeeded in building calculating engines.
- These used mechanical relays but were very slow, with cycle times measured in seconds. Relays were later replaced by vacuum tubes. These machines were enormous, filling up entire rooms with tens of thousands of vacuum tubes, but they were still millions of times slower than even the cheapest personal computers available today.

- All programming was done in absolute machine language, often by wiring up plug boards to control the machine's basic functions.

1.2 The Second Generation (1955-65) Transistors and Batch Systems:

- The transistor were introduced in the mid-1950s leading to the development of mainframes.
- Mainframes were placed in specially air conditioned computer rooms, with staffs of professional operators to run them.
- Only big corporations or major government agencies or universities were able to afford to buy because of the high cost.
- To run a program or set of programs, a programmer would first write the program on paper (in FORTRAN or assembler), then punch it on cards. These cards were placed in mainframe for getting the inputs processed. The below figure describes this system where inputs and outputs processed by mainframe are individually collected.

- Next came the batch system, where jobs were collected in a single room and then read onto a magnetic tape using inexpensive computers. After jobs getting processed, the outputs are written onto a second tape.
- After each job finished, the operating system automatically read the next job from the tape and began running it.
- When the whole batch was done, the operator removed the input and output tapes, replaced the input tape with the next batch, and brought the output tape to a 1401 for printing offline

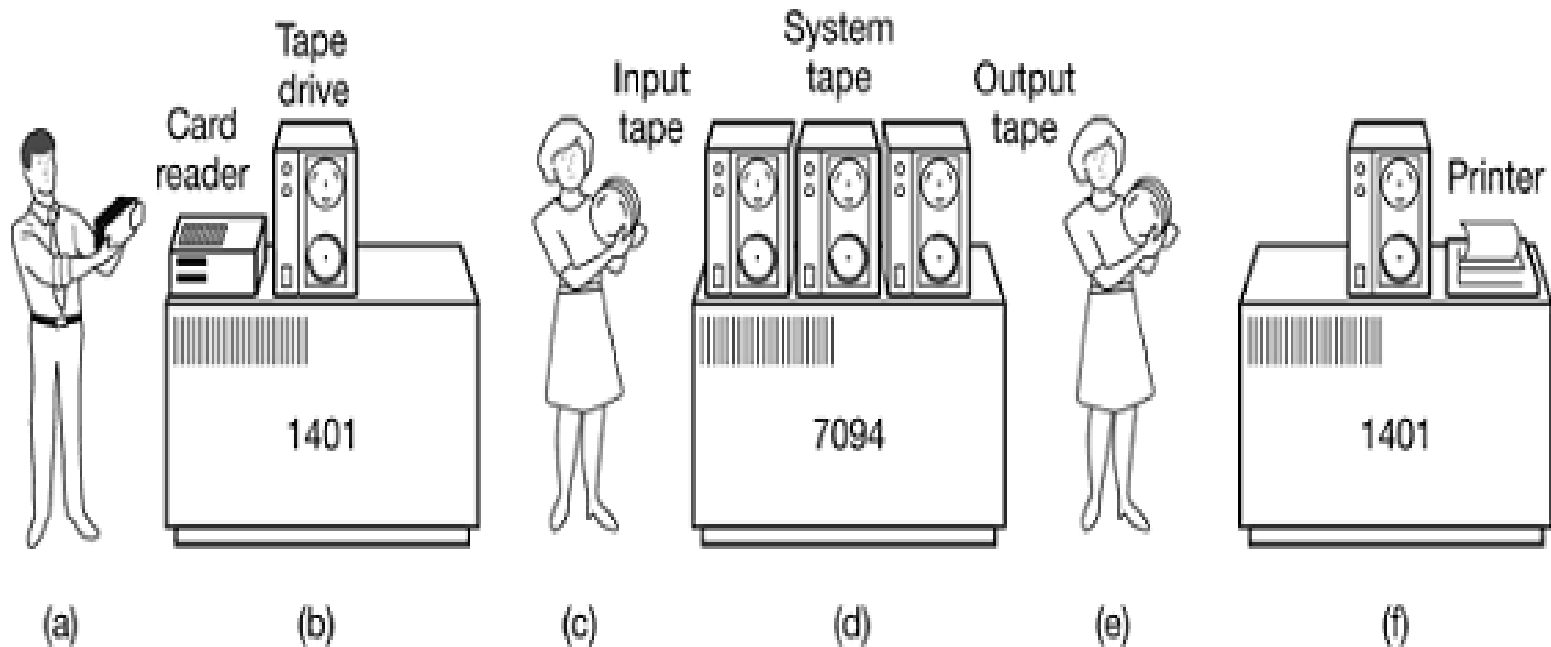
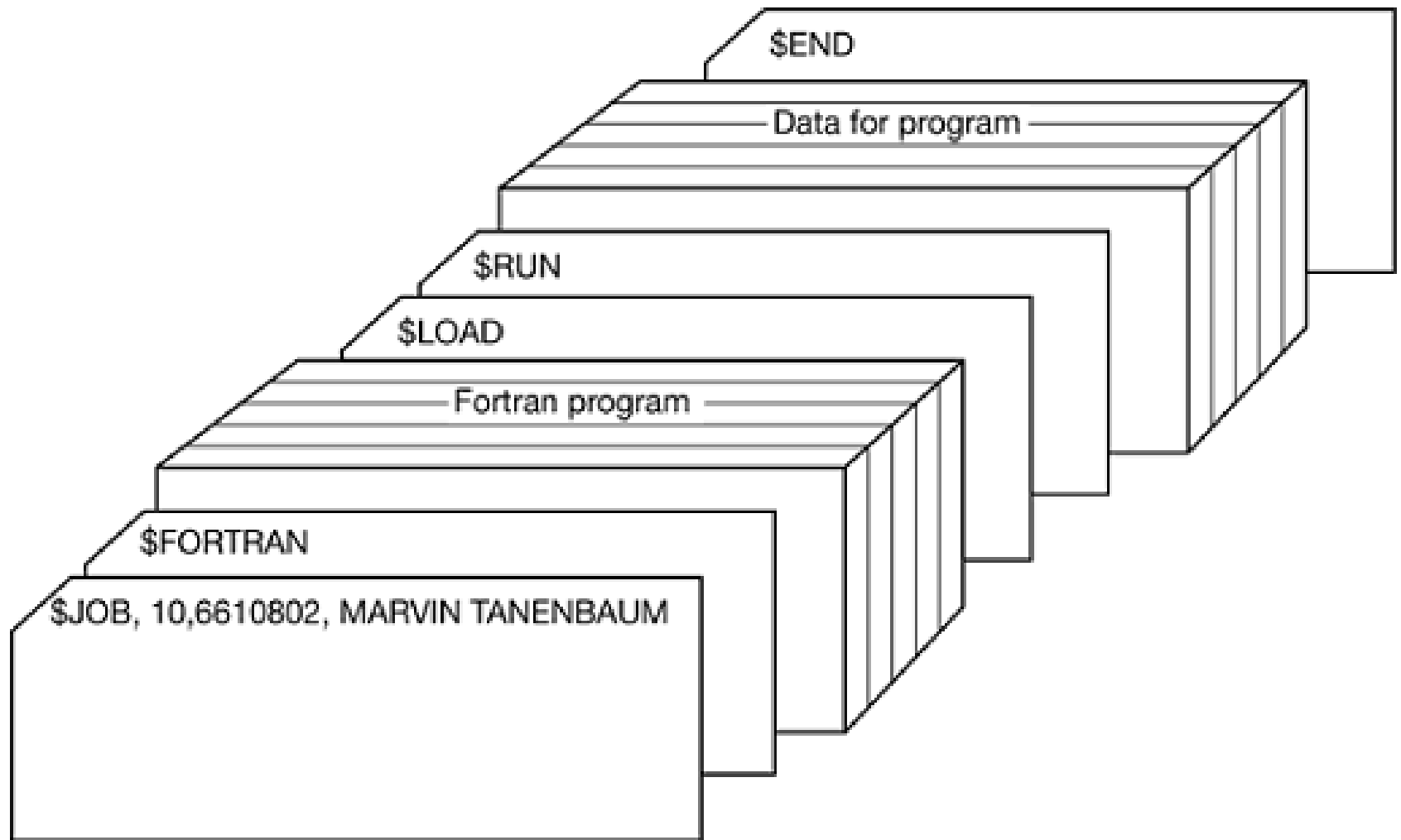


FIG 1: An early batch system.

- (a) Programmers bring cards to 1401.
- (b) 1401 reads batch of jobs onto tape.
- (c) Operator carries input tape to 7094.
- (d) 7094 does computing.
- (e) (e) Operator carries output tape to 1401. (f) 1401 prints output.

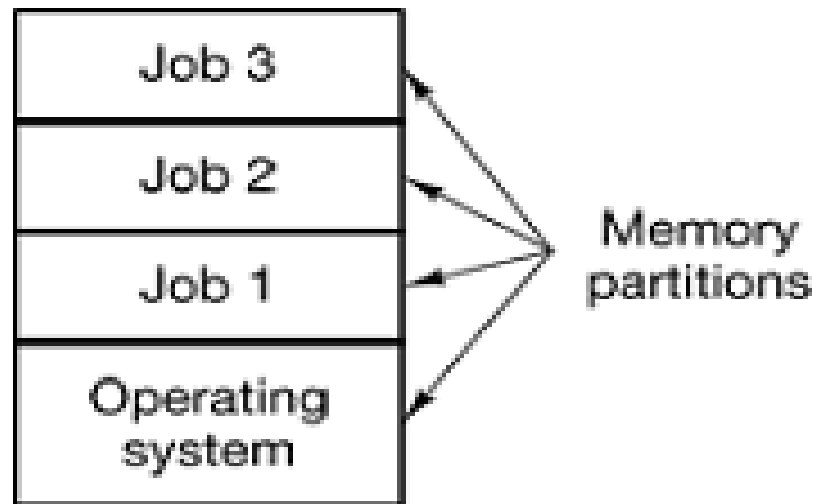
- The structure of a typical input job is shown in below figure,
 - The \$JOB card specifies the maximum run time in minutes, the account number to be charged, and the programmer's name.
 - \$FORTRAN card tells the operating system to load the FORTRAN compiler from the system tape. Then the program to be compiled is loaded.
 - \$LOAD card directs the operating system to load the object program just compiled.
 - \$RUN card, tells the operating system to run the program with the data following it.
 - The \$END card marked the end of the job
- Large second-generation computers were used mostly for scientific and engineering calculations, such as solving the partial differential equations.
- They were largely programmed in FORTRAN and assembly language. Typical operating systems were FMS (the Fortran Monitor System) and IBSYS, IBM's operating system for the 7094



Structure of a typical FMS job

- **1.3 The Third Generation (1965-1980) ICs and Multiprogramming:**
- The IBM's 360 was the first major computer line to use (small-scale) Integrated Circuits (ICs) providing a major performance advantage over the second-generation machines, which were built up from individual transistors.
- Nowadays they are often used for managing huge databases (e.g., for airline reservation systems) or as servers for World Wide Web sites that must process thousands of requests per second.
- To meet the requirements like compatibility with all models, small systems, large systems, work capability in commercial and scientific environments, the operating system OS/360 was developed with millions of lines of assembly language written by thousands of programmers.
- The most important of these requirements was multiprogramming.
- On the 7094, when the current job paused to wait for a tape or other I/O operation to complete, the CPU simply sat idle until the I/O finished. With heavily CPU-bound scientific calculations, I/O is infrequent, so this wasted time is not significant. With commercial data processing, the I/O wait time can often be 80 or 90 percent of the total time, so something had to be done to avoid having the (expensive) CPU be idle so much

- The solution that evolved was to partition memory into several pieces with a different job in each partition, as shown in below diagram.
- While one job was waiting for I/O to complete, another job could be using the CPU. If enough jobs could be held main memory at once, the CPU could be kept busy nearly 100 percent of the time.
- The 360 and other third-generation systems were equipped with special hardware to protect each job residing in main memory against snooping and mischief by the other ones,



A multiprogramming system with three jobs in memory

- Another ability of third generation operating system was spooling. Spooling means the ability to store jobs onto disk and do the processing on jobs and later store the processed job onto disk, so that they can be read from the disk whenever needed.
- desire for quick response time paved the way for timesharing, a variant of multiprogramming,
- The first serious timesharing system, CTSS (Compatible Time Sharing System), was developed at M.I.T. on a specially modified 7094
- Later a machine called MULTICS (MULTiplexed Information and Computing Service), that would support hundreds of simultaneous timesharing users was created. Their model was the electricity distribution system — when you need electric power, you just stick a plug in the wall, and within reason, as much power as you need will be there.
- Another major development during the third generation was the phenomenal growth of minicomputers, starting with the DEC PDP-1 in 1961. The PDP-1 had only 4K of 18-bit words

- It was quickly followed by a series of other PDPs, at last the development of PDP series ended with the development of the PDP-11.
- One of the computer scientists at Bell Labs wrote a one-user version of MULTICS. Which later developed into the UNIX operating system.
- Due to source code availability many different versions were created with different compatibility.
- But To make it possible to write programs that could run on any UNIX system. IEEE developed a standard for UNIX, called POSIX, that most versions of UNIX now support. POSIX defines a minimal system call interface that all UNIX systems must support.
- In 1987, a small clone of UNIX, called MINIX, for educational purposes including support was released.
- Finnish student, Linus Torvalds developed Linux, a free production version with many MINIX features and having common structure as MINIX and UNIX

1.4 The Fourth Generation (1980-Present) Personal Computers:

- With the development of LSI (Large Scale Integration) circuits, chips containing thousands of transistors on a square centimeter of silicon, the age of the personal computer dawned. In terms of architecture, personal computers were not all that different from minicomputers of the PDP-11 class. The microprocessor chip made it possible for a single individual to have his or her own personal computer.
- In 1974, after Intel came out with the 8080, the first general-purpose 8-bit CPU, Gary Kildall developed a controller for 8 inch floppy disk and hooked the floppy disk up to the 8080, thus producing the first microcomputer with a disk. Kildall then wrote a disk-based operating system called CP/M (Control Program for Microcomputers) and formed a company “Digital Research”
- In 1977, Digital Research rewrote CP/M to make it suitable for running on the many microcomputers using the 8080, Zilog Z80, and other CPU chips. Many application programs were written to run on CP/M, allowing it to completely dominate the world of micro computing for about 5 years.

- In 1980s, IBM designed the IBM PC and was searching for software to run on it.
- People from IBM contacted Bill Gates to license his BASIC interpreter. After licensing, Bill Gates suggested operating system CP/M. But due to some misunderstandings, Digital Research refused to IBM's offers. Then IBM asked Gates to develop an operating system.
- Gates procured DOS (Disk operating system) from Seattle Computer Products. Gates told the IBM that he would give DOS/BASIC package. Tim Paterson (DOS creator) hired by Gates did some modifications to DOS and the revised system was named as MS-DOS (Microsoft disk operating system).
- IBM PC/AT came out in 1983 with the Intel 80286 CPU and MS-DOS. The later versions of MS-DOS on 80386 and 80486 had more features taken from UNIX.
- So far CP/M, MS-DOS and other operating systems were all based on users typing in commands from the keyboard.
- Doug Engelbart at Stanford Research Institute invented the GUI (Graphical User Interface), pronounced "gooey," complete with windows, icons, menus, and mouse. These ideas were adopted by researchers at Xerox PARC and incorporated into machines they built.

- Steve Jobs, who co-invented the Apple computer visited PARC, saw a GUI, and instantly realized its potential value.
- Then Steve jobs started building an Apple computer with a GUI called LISA, which was very expensive and so failed to be sold in markets.
- Later the Apple Macintosh was invented, which was a huge success, not only because it was much cheaper than the Lisa, but also because it was user friendly, meaning that it was intended for users who not only knew nothing about computers.
- When Microsoft decided to build a successor to MS-DOS, Microsoft influenced by Macintosh success produced a GUI-based system called Windows, which originally ran on top of MS-DOS.
- From 1985 to 1993, Windows was just a graphical environment on top of MS-DOS. However, starting in 1995 a freestanding version of Windows, Windows 95, was released that incorporated many operating system features into it, using the underlying MS-DOS system only for booting and running old MS-DOS programs,
- In 1998, a slightly modified version of this system, called Windows 98 was released. Windows 95 and 98 were 16 bit operating system.

- Another Microsoft operating system is Windows NT was developed and is a 32 bit system.
- Version 5 of Windows NT was renamed Windows 2000 in early 1999. It was intended to be the successor to both Windows 98 and Windows NT 4.0. That did not quite work out either, so Microsoft came out with yet another version of Windows 98 called Windows Me (Millennium edition).
- The other major contender in the personal computer world is UNIX . UNIX is strongest on workstations and other high-end computers, such as network servers. It is especially popular on machines powered by high-performance RISC chips.
- On Pentium-based computers, Linux is becoming a popular alternative to Windows for students and increasingly many corporate users
- during the mid-1980s, the growth of networks of personal computers running network operating systems and distributed operating systems became popular.

- In a network operating system, the users are aware of the existence of multiple computers and can log in to remote machines and copy files from one machine to another. Each machine runs its own local operating system and has its own local user. Network operating systems are not fundamentally different from single-processor operating systems. They obviously need a network interface controller and some low-level software to drive it, as well as programs to achieve remote login and remote file access,
- A distributed operating system, in contrast, is one that appears to its users as a traditional uniprocessor system, even though it is actually composed of multiple processors. The users should not be aware of where their programs are being run or where their files are located; that should all be handled automatically and efficiently by the operating system.

2. THE OPERATING SYSTEM ZOO

2.1 Mainframe Operating Systems:

- Mainframes are room-sized computers still found in major corporate data centers with high I/O capacity and high amount of data storage capacity. An example mainframe operating system is OS/390, a descendant of OS/360.
- The operating systems for mainframes are heavily oriented toward processing many job at once, most of which need prodigious amounts of I/O. They typically offer three kinds of services: batch, transaction processing, and timesharing.
- A batch system is one that processes routine jobs without any interactive user present. Ex. processing in an insurance company or sales reporting for a chain of stores is typically done in batch mode.
- Transaction processing systems handle large numbers of small requests, for example, check processing at a bank or airline reservations. Each unit of work is small, but the system must handle hundreds or thousands per second.
- Timesharing systems allow multiple remote users to run jobs on the computer at once, such as querying a big database.

2.2 Server Operating Systems:

- These run on servers, which are either very large personal computers, workstations, or even mainframes. They serve multiple users at once over a network and allow the users to share hardware and software resources. Servers can provide print service, file service, or Web service. Internet providers run many server machines to support their customers and Web sites use servers to store the Web pages and handle the incoming requests. Typical server operating systems are UNIX and Windows 2000. Linux is also gaining ground for servers.

2.3 Multiprocessor Operating Systems:

- In order to attain higher computing power the idea is to connect multiple CPUs into a single system.
- Depending on precisely how they are connected and what is shared, these systems are called parallel computers, multicomputer, or multiprocessors. They need special operating systems, but often these are variations on the server operating systems, with special features for communication and connectivity.

2.4 Personal Computer Operating Systems:

- Their job is to provide a good interface to a single user. They are widely used for word processing, spreadsheets, and Internet access. Common examples are Windows 98, Windows 2000, the Macintosh operating system, and Linux.
- Real-Time Operating Systems
- Real time operating systems focusses not on user friendly interface, rather they focusses on completing some job/process in a exact time in which the job has to be completed. For example, if a car is moving down an assembly line, certain actions must take place at certain instants of time, if a welding robot, welds too early or too late, the car will be ruined.
- So hard real time system are systems in which deadline cannot be missed, If deadlines are missed it leads to catastrophic.
- Another kind of real-time system is a soft real-time system, in which missing an occasional deadline is acceptable. Digital audio or multimedia systems fall in this category.
- VxWorks and QNX are well-known real-time operating systems.

2.5 Embedded Operating Systems:

- Embedded systems run on the computers that control devices that are not generally thought of as computers, such as TV sets, microwave ovens, and mobile telephones. These often have some characteristics of real-time systems but also have size, memory, and power restrictions that make them special. Examples of such operating systems are PalmOS and Windows CE (Consumer Electronics)

2.6 Smart Card Operating Systems

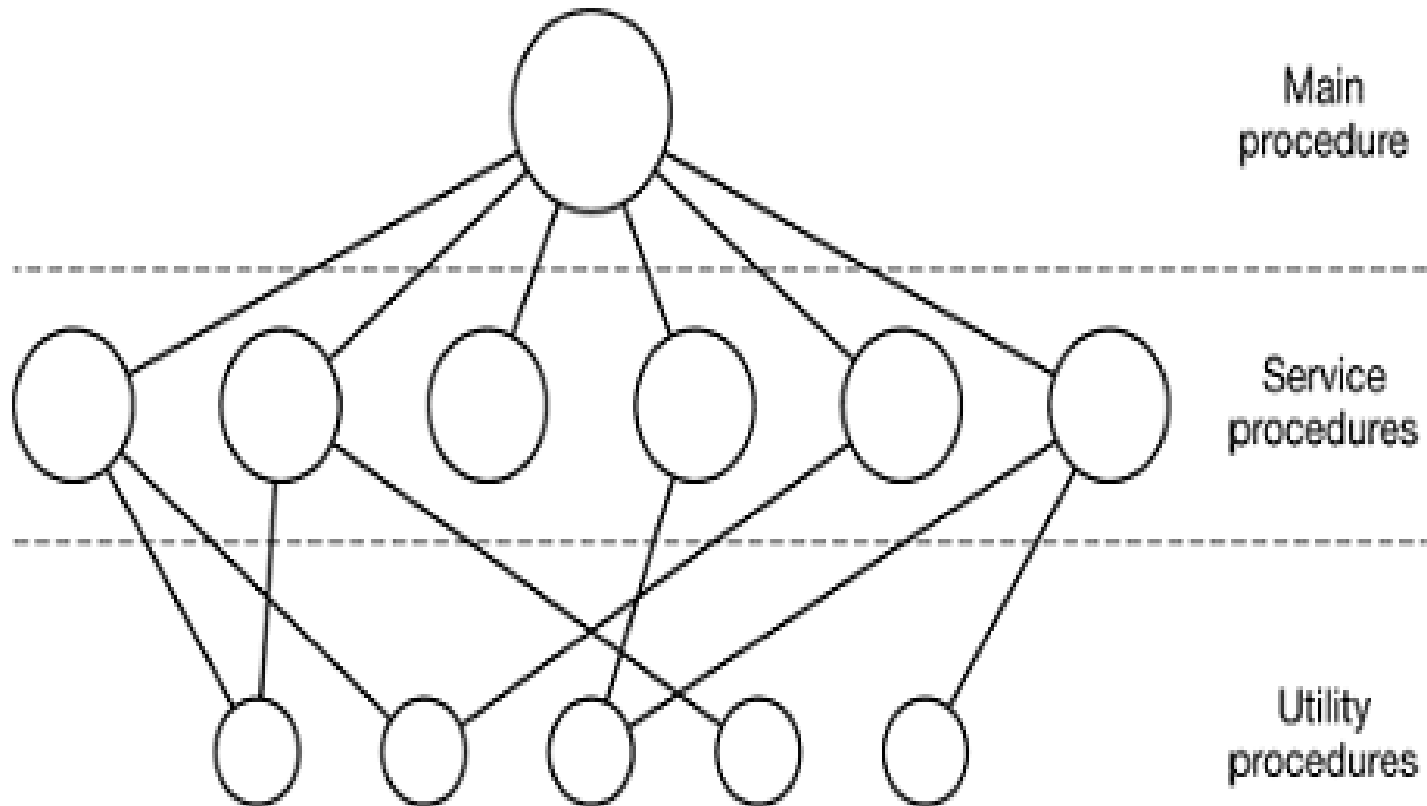
- The smallest operating systems run on smart cards.
- Smart cards are credit card-sized devices containing a CPU chip. They have very severe processing power and memory constraints. Some of them can handle only a single function, such as electronic payments, but others can handle multiple functions on the same smart card.
- Some smart cards are Java oriented. ROM on the smart card holds an interpreter for the Java Virtual Machine (JVM).

- Java applets (small programs) are downloaded to the card and are interpreted by the JVM interpreter. Some of these cards can handle multiple Java applets at the same time, leading to multiprogramming and the need to schedule them. Resource management and protection also become an issue when two or more applets are present at the same time.

3. OPERATING SYSTEM STRUCTURE:

- five different operating system structures that exists till today are,

3.1 Monolithic system:



monolithic system structure

- In monolithic systems, The operating system is written as a collection of procedures, each of which can call any of the other ones whenever needed.
- When this technique is used, each procedure in the system has a well-defined interface in terms of parameters and results, and each one is free to call any other one.
- To construct the actual object program of the operating system, one first compiles all the individual procedures, or files containing the procedures, and then binds them all together into a single object file using the system linker.
- There is no information hiding, as every procedures are visible to every other procedure
- In monolithic system, It is possible to have at least a little structure.
- The services (system calls) provided by the operating system are requested by putting the parameters in a well-defined place (e.g., on the stack) and then executing a trap instruction. This Trap instruction switches the machine from user mode to kernel mode and transfers control to the operating system,

- The operating system then fetches the parameters and determines which system call is to be carried out. After that, it indexes into a table that contains in slot k- a pointer to the procedure that carries out system call k.
- Here the basic structure for the operating system is,
 1. A main program invokes the requested service procedure.
 2. A set of service procedures that carry out the system calls..
 3. A set of utility procedures that help the service procedures.

3.2 Layered Systems:

- Here operating system is organized as a hierarchy of layers, each one constructed upon the one below it. The first system constructed in this way was the THE system built at the Technische Hogeschool Eindhoven in the Netherlands by E. W. Dijkstra (1968) and his students.
- The THE system was a simple batch system for a Dutch computer, the Electrologica X8.

- The system had 6 layers, as shown in below diagram,

The operator

4

User programs

3

Input/output management

2

Operator-process communication

1

Memory and drum management

0

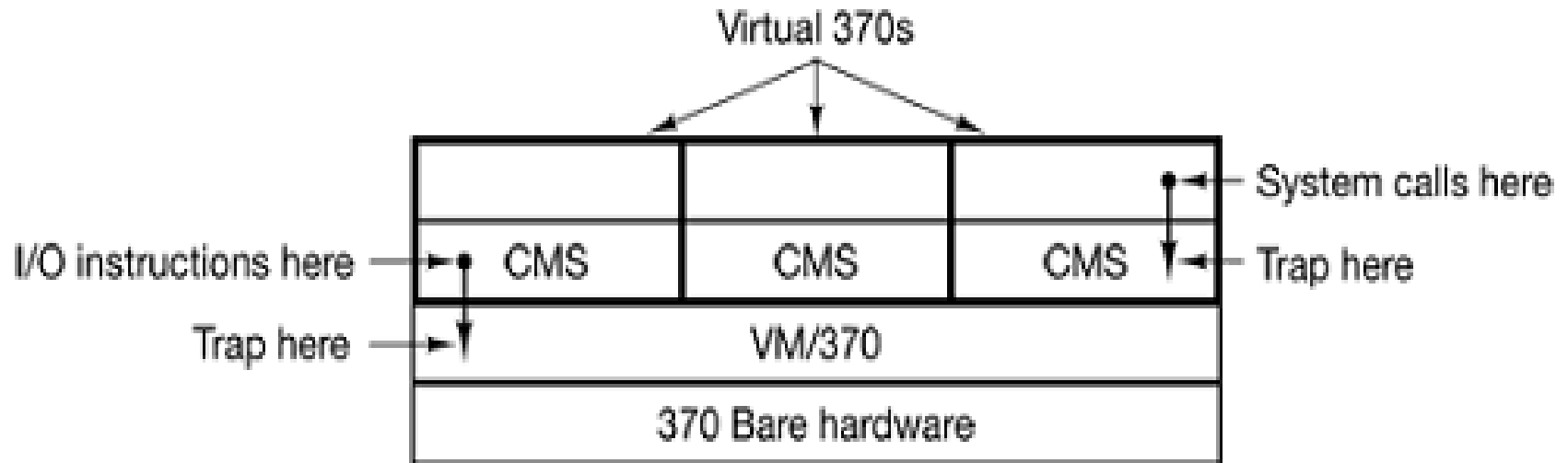
Processor allocation and multiprogramming

Structure of THE operating system

- Layer 0 dealt with allocation of the processor, switching between processes when interrupts occurred or timers expired and it also provided the basic multiprogramming of the CPU.
- Layer 1 does the memory management.
- It allocated space for processes in main memory and also on a 512K word drum used for holding parts of processes (pages) for which there was no room in main memory.
- Layer 2 handled communication between each process and the operator console. Above this layer each process effectively had its own operator console.
- Layer 3 took care of managing the I/O devices and buffering the information streams to and from them.
- Layer 4 contains user programs. They did not have to worry about any functions like process, memory, console, or I/O management as they are taken care by lower layers.
- The system operator process was located in layer 5.

3.3 Virtual Machines

- A group at IBM's Scientific Center in Cambridge, Massachusetts, produced a radically different time sharing system that IBM accepted as a product and is now widely used on its mainframes. This system was called CP/CMS and later renamed as VM/370.
- A timesharing system provides (1) multiprogramming and (2) an extended machine with a more convenient interface than the bare hardware.
- Here The heart of the system, known as the virtual machine monitor, runs on the bare hardware and does the multiprogramming and it provides several virtual machines to the next layer up, as shown in below figure.
- However, unlike all other operating systems, these virtual machines are not extended machines, with files and other nice features. Instead, they are exact copies of the bare hardware, including kernel/user mode, I/O, interrupts, and everything else the real machine has.



- Because each virtual machine is identical to the true hardware, each one can run any operating system that will run directly on the bare hardware.
- Different virtual machines can, and frequently do, run different operating systems
- Some run one of the descendants of OS/360 for batch or transaction processing, while other ones run a single-user, interactive system called CMS (Conversational Monitor System) for interactive timesharing users.

- When a CMS program executes a system call, the call is trapped to the operating system in its own virtual machine, not to VM/370, just as it would if it were running on a real machine instead of a virtual one.
- CMS then issues the normal hardware I/O instructions for reading its virtual disk or whatever is needed to carry out the call.
- These I/O instructions are trapped by VM/370, which then performs them as part of its simulation of the real hardware. By completely separating the functions of multiprogramming and providing an extended machine.
- Intel provided a virtual 8086 mode on the Pentium. In this mode, the machine acts like an 8086 (which is identical to an 8088 from a software point of view), including 16-bit addressing with a 1-MB limit.
- This mode is used by Windows and other operating systems for running MS-DOS programs. These programs are started up in virtual 8086 mode. As long as they execute normal instructions, they run on the bare hardware. However, when a program tries to trap to the operating system to make a system call, or tries to do protected I/O directly, a trap to the virtual machine monitor occurs.

Two variants on this design are possible:

- First one, MS-DOS itself is loaded into the virtual 8086's address space, so the virtual machine monitor just reflects the trap back to MS-DOS, just as would happen on a real 8086. When MS-DOS later tries to do the I/O itself, that operation is caught and carried out by the virtual machine monitor.
- In the other variant, the virtual machine monitor just catches the first trap and does the I/O itself, since it knows what all the MS-DOS system calls are and thus knows what each trap is supposed to do. This variant is less pure than the first one, since it only emulates MS-DOS correctly, and not other operating systems, as the first one does.
- On the other hand, it is much faster, since it saves the trouble of starting up MS-DOS to do the I/O.
- A further disadvantage of actually running MS-DOS in virtual 8086 mode is that MS-DOS fiddles around with the interrupt enable/disable bit quite a lot, all of which must be emulated at considerable cost.

- Another area where virtual machines are used, but in a somewhat different way, is for running Java programs. When Sun Microsystems invented the Java programming language, it also invented a virtual machine (i.e., a computer architecture) called the JVM (Java Virtual Machine). The Java compiler produces code for JVM, which then typically is executed by a software JVM interpreter. The advantage of this approach is that the JVM code can be shipped over the Internet to any computer that has a JVM interpreter and run there.
- Another advantage of using JVM is that if the interpreter is implemented properly, which is not completely trivial, incoming JVM programs can be checked for safety and then executed in a protected environment so they cannot steal data or do any damage.

3.4 Exokernels:

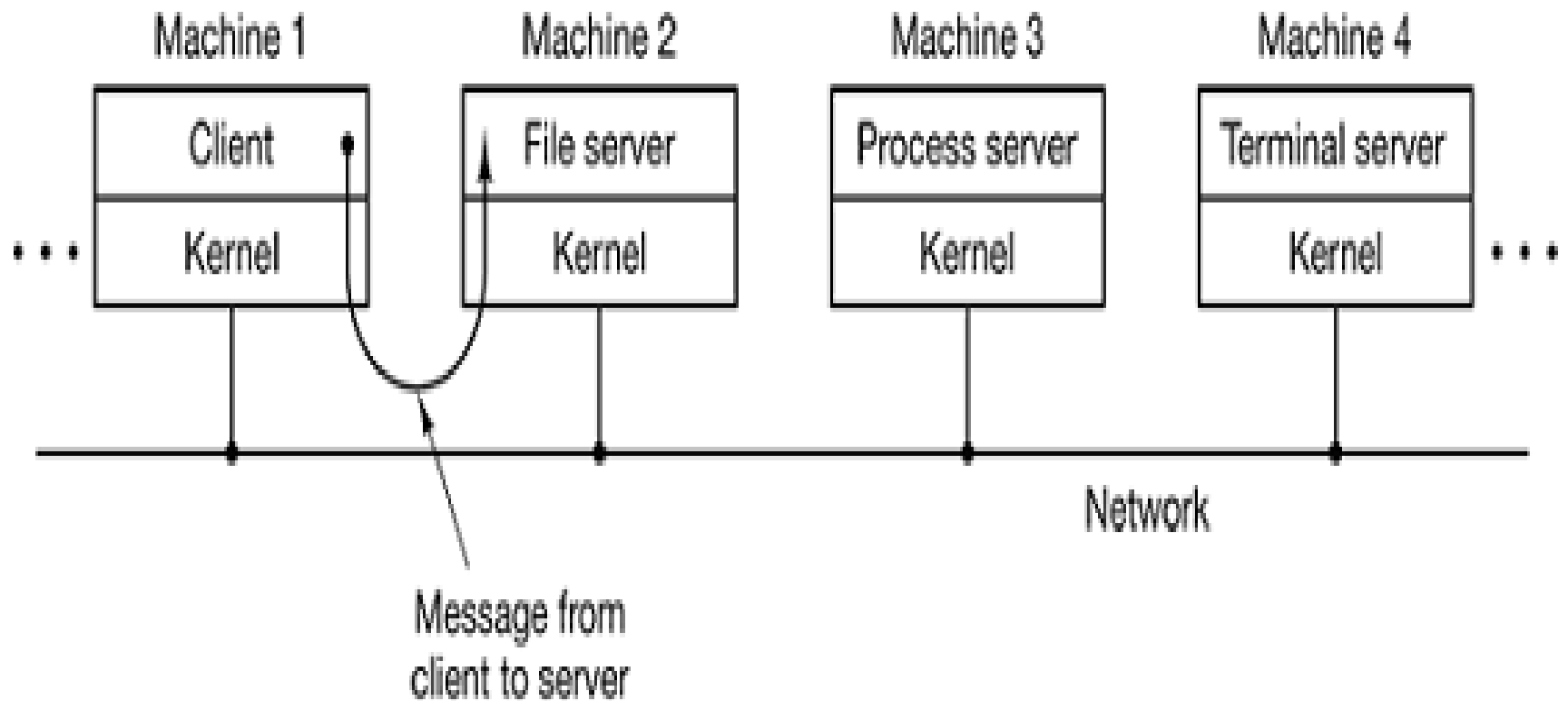
- With VM/370, each user process gets an exact copy of the actual computer. With virtual 8086 mode on the Pentium, each user process gets an exact copy of a different computer. Going one step further, researchers at M.I.T. have built a system that gives each user a clone of the actual computer, but with a subset of the resources

- Thus one virtual machine might get disk blocks 0 to 1023, the next one might get blocks 1024 to 2047, and so on
- At the bottom layer, running in kernel mode, is a program called the exokernel. Its job is to allocate resources to virtual machines and then check attempts to use them to make sure no machine is trying to use somebody else's resources. Each user-level virtual machine can run its own operating system, as on VM/370 and the Pentium virtual 8086s, except that each one is restricted to using only the resources it has asked for and been allocated
- The advantage of the exokernel scheme is that it saves a layer of mapping. In the other designs, each virtual machine thinks it has its own disk, with blocks running from 0 to some maximum, so the virtual machine monitor must maintain tables to remap disk addresses (and all other resources). With the exokernel, this remapping is not needed. The exokernel need only keep track of which virtual machine has been assigned which resource. This method still has the advantage of separating the multiprogramming (in the exokernel) from the user operating system code (in user space), but with less overhead, since all the exokernel has to do is keep the virtual machines out of each other's hair.

3.5 Client-Server Model:

- A trend in modern operating systems is to take the idea of moving code up into higher layers and removing codes as much as possible from kernel mode, leaving a minimal microkernel.
- The usual approach is to implement most of the operating system in user processes. To request a service, such as reading a block of a file, a user process (now known as the client process) sends the request to a server process, which then does the work and sends back the answer.

- In the above shown model, the kernel handles the communication between clients and servers. By splitting the operating system up into parts, each of which only handles one facet of the system, such as file service, process service, terminal service, or memory service, each part becomes small and manageable. Furthermore, because all the servers run as user-mode processes, and not in kernel mode, they do not have direct access to the hardware. As a consequence, if a bug in the file server is triggered, the file service may crash, but this will not usually bring the whole machine down.
- Another advantage of the client-server model is its adaptability to use in distributed systems (see Fig. 1-28). If a client communicates with a server by sending it messages, the client need not know whether the message is handled locally in its own machine, or whether it was sent across a network to a server on a remote machine. As far as the client is concerned, the same thing happens in both cases: a request was sent and a reply came back.



The client server model in distributed system